

Tìm số chiều để giữ 95% phương sai

Chạy PCA mà không giảm chiều để lấy toàn bộ các PCs, tính toán lũy kế:

```
1 from sklearn.datasets import fetch_openml
2 import numpy as np
3
4 mnist = fetch_openml('mnist_784', as_frame=False)
5 X_train = mnist.data[:60_000]
6
7 pca = PCA()
8 pca.fit(X_train)
9
10 cumsum = np.cumsum(pca.explained_variance_ratio_)
11 d = np.argmax(cumsum >= 0.95) + 1
12 # d bằng 154
13
```

Đặt `n_components` là tỷ lệ phần trăm

Một cách tốt hơn (và nhàn hơn) là truyền thẳng mức độ phương sai bạn muốn bảo toàn dưới dạng một số thập phân từ 0.0 đến 1.0 cho `n_components`:

```
1 # Scikit-learn sẽ tự tính toán số chiều tối thiểu
2 pca = PCA(n_components=0.95)
3
4 X_reduced = pca.fit_transform(X_train)
5
6 print(pca.n_components_)
7 # Output: 154
8
```

Vẽ biểu đồ phương sai giải thích theo số chiều

Một phương pháp trực quan là vẽ mảng `cumsum` (phương sai tích lũy).

(Giả lập Hình 8-8: Biểu đồ đường tăng dần và tạo thành một điểm uốn - elbow point - khi đạt $\approx 100 - 150$ chiều, sau đó đường đi ngang vì các chiều thêm vào không mang thêm phương sai.)

Tại "điểm uốn" (elbow), việc tăng thêm số chiều không mang lại quá nhiều giá trị thông tin, giúp người phân tích dễ dàng quyết định số chiều cần thiết.

Điều chỉnh số chiều như một siêu tham số

- Nếu bạn sử dụng PCA như một bước *tiền xử lý (preprocessing)* cho một tác vụ Machine Learning khác (ví dụ Rừng ngẫu nhiên), bạn có thể coi số chiều d như một **siêu tham số (Hyperparameter)**.
- Có thể dùng `GridSearchCV` hoặc `RandomizedSearchCV` để tìm số chiều tốt nhất cho bài toán đó.

Sử dụng RandomizedSearchCV cho PCA

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3 from sklearn.pipeline import make_pipeline
4
5 clf = make_pipeline(PCA(random_state=42), RandomForestClassifier(random_state
    =42))
6 param_distrib = {
7     "pca__n_components": np.arange(10, 80),
8     "randomforestclassifier__n_estimators": np.arange(50, 500)
9 }
10 rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
    random_state=42)
11 rnd_search.fit(X_train[:1000], y_train[:1000])
12
13 print(rnd_search.best_params_)
14 # pca__n_components tối ưu chỉ là 23 thay vì 154
15
```

PCA dùng để nén dữ liệu (Compression)

- Sau khi áp dụng PCA bảo toàn 95% phương sai, MNIST đã giảm từ 784 đặc trưng xuống còn 154.
- Tập dữ liệu lúc này chỉ còn bằng $\approx 20\%$ kích thước ban đầu, trong khi chỉ **mất 5% thông tin!**
- Mức độ nén tuyệt vời này giúp tiết kiệm bộ nhớ RAM và tăng tốc vượt trội thuật toán học máy phía sau.

Tái tạo dữ liệu (Giải nén)

- Bạn có thể *giải nén* tập dữ liệu 154 chiều ngược trở lại thành 784 chiều.
- Tuy nhiên, đây là nén **có hao hụt (lossy compression)**, bạn sẽ không nhận lại được hoàn toàn 100% pixel gốc (thiếu 5% phương sai đã loại bỏ).
- Hình ảnh tái tạo sẽ bị mờ đi một chút, nhưng phần lớn nội dung gốc vẫn nguyên vẹn.
- *Sai số tái tạo (Reconstruction Error)*: Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu giải nén.

Phép biến đổi ngược (inverse_transform)

Sử dụng `inverse_transform` trên đối tượng PCA:

```
1 X_recovered = pca.inverse_transform(X_reduced)
```

(Giả lập Hình 8-9: Hình ảnh MNIST gốc so sánh với hình ảnh đã qua nén/giải nén bằng PCA. Hình giải nén tuy không sắc nét hoàn hảo bằng nhưng vẫn dễ dàng đọc được là số mấy).

PCA ngẫu nhiên (Randomized PCA)

- Thuật toán **Randomized PCA** dùng kỹ thuật ngẫu nhiên hóa để xấp xỉ tìm d thành phần chính đầu tiên một cách nhanh chóng.
- Độ phức tạp tính toán: $O(m \times d^2) + O(d^3)$, thay vì $O(m \times n^2) + O(n^3)$ của SVD tiêu chuẩn.
- Nó nhanh hơn đáng kể so với PCA đầy đủ khi $d \ll n$ (số chiều giảm xuống nhỏ hơn nhiều số chiều ban đầu).

Cài đặt Randomized PCA

Chỉ định tham số `svd_solver="randomized"`:

```
1 rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
2
3 X_reduced = rnd_pca.fit_transform(X_train)
4
```

PCA tăng dần (Incremental PCA)

- **Vấn đề của PCA thông thường:** Yêu cầu TOÀN BỘ tập dữ liệu phải nằm gọn trong bộ nhớ (RAM) cùng một lúc.
- **Giải pháp:** Incremental PCA (IPCA) cho phép bạn chia tập huấn luyện thành các *mini-batch* và đẩy chúng vào mô hình từng lô nhỏ một.
- Hữu ích cho các tập dữ liệu khổng lồ (Out-of-core learning) và để áp dụng PCA theo thời gian thực (Online).

Cài đặt Incremental PCA với array_split

Lưu ý gọi phương thức `partial_fit()` ở từng bước thay vì `fit()`:

```
1 from sklearn.decomposition import IncrementalPCA
2 import numpy as np
3
4 n_batches = 100
5 inc_pca = IncrementalPCA(n_components=154)
6
7 for X_batch in np.array_split(X_train, n_batches):
8     inc_pca.partial_fit(X_batch)
9
10 # transform áp dụng cho toàn bộ
11 X_reduced = inc_pca.transform(X_train)
12
```

Tối ưu Incremental PCA với tập bộ nhớ (mmap)

Sử dụng `np.mmap` để sử dụng file nhị phân trên đĩa như thể nó nằm trong RAM. Dữ liệu chỉ tải vào RAM khi cần:

```
1 # doc file đã lưu dưới dạng mmap
2 filename = "my_mnist.mmap"
3 X_mmap = np.mmap(filename, dtype="float32", mode="readonly").reshape(-1, 784)
4
5 batch_size = X_mmap.shape[0] // n_batches
6 inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
7
8 # inc_pca hiểu cách hoạt động của mmap và sẽ lấy từng đoạn dữ liệu theo
   batch_size
9 inc_pca.fit(X_mmap)
10
```

Phép chiếu ngẫu nhiên (Random Projection)

- Chiếu dữ liệu xuống không gian thấp bằng một **ma trận ngẫu nhiên**.
- Thay vì dùng SVD phức tạp, Random Projection chứng minh rằng một ma trận chiếu tạo ngẫu nhiên *rất có khả năng bảo toàn khoảng cách* giữa các điểm dữ liệu.
- Ưu điểm: **Không cần huấn luyện!** Chỉ phụ thuộc vào hình dạng ma trận ban đầu. Rất nhẹ và cực nhanh cho dữ liệu chiều cực lớn (hàng chục nghìn features).

- **Định lý Johnson-Lindenstrauss (JL):** Có thể chiếu m điểm từ không gian chiều cao xuống không gian d chiều sao cho khoảng cách bình phương giữa mọi cặp điểm chỉ bị méo mó tối đa là một sai số ε .
- Ví dụ: Bạn có 5.000 điểm (m), với dung sai $\varepsilon = 0.1$.
- Định lý tính được $d \approx 7300$ chiều là đủ an toàn. Phương trình này **không quan tâm đến số chiều gốc n** (ngay cả khi $n = 1$ triệu).